

TESCHA

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA

Programación de Aplicaciones para Dispositivos Móviles

PROFESOR

Iván Azamar Palma

ALUMNO

Yañez Moreno Alexis Gabriel

GRUPO:

4952



	INGENIERÍA EN SISTEMAS COMPUTACIONALES	
---	---	---

DATOS GENERALES	
ASIGNATURA: Programación en Tecnologías Web para la Industria	
TÍTULO DE LA PRÁCTICA: Reporte de Practicas Tema 2	
DOCENTE: Iván Azamar Palma	FECHA: 19-10-2025
ESTUDIANTE(S) Yañez Moreno Alexis Gabriel	GRUPO: 4952

OBJETIVO DE LA PRÁCTICA: Implementar el almacenamiento y recuperación de datos primitivos (String, Int, Boolean) usando SharedPreferences.	
COMPETENCIA(S) ESPECÍFICA(S): Desarrolla aplicaciones para aplicaciones web incluyendo	COMPETENCIA(S) GENÉRICA(S): Crea y aplica funcionalidades especiales para aplicaciones de comercio electrónico como proyecto final.

REQUERIMIENTOS	
FÓRMULAS/TÉCNICAS/PROCESOS/PROCEDIMIENTOS:	
RECURSOS MATERIALES: Laboratorio de cómputo con internet.	RECURSOS TÉCNICOS/TECNOLÓGICOS: - Android Studio (versión actual) - Kotlin - Dispositivo físico o emulador Android

Contenido

Marco Teórico.....	4
1. Android Studio:.....	4
2. Kotlin.....	4
3. Optional Chaining.....	4
4. Nullish Coalescing Operator (Elvis).....	4
5. Listas mutables e inmutables	4
6. Operaciones sobre listas.....	5
7. Sobrecarga de funciones	5
8. Lambda	5
9. forEach	5
10. vararg	6
11. Excepciones.....	6
Desarrollo	7
PRÁCTICA 1.....	7
PRÁCTICA 2:	12
PRÁCTICA 3:	17
Resultados	25
Practica1:	25
Practica2:	25
Practica3:	26
Conclusión general:.....	30
Referencias bibliográficas:	31

Marco Teórico

1. Android Studio:

Android Studio es el **entorno de desarrollo integrado (IDE)** oficial para crear aplicaciones móviles para Android. Basado en IntelliJ IDEA, ofrece herramientas para la edición de código, depuración, pruebas y empaquetado de aplicaciones. Permite trabajar con **lenguajes como Kotlin y Java**, facilita el diseño de interfaces mediante un editor visual y genera automáticamente archivos de configuración para el sistema operativo Android. Además, incluye un **emulador** para probar aplicaciones sin necesidad de un dispositivo físico.

2. Kotlin

Kotlin es un **lenguaje de programación moderno, conciso y seguro**, desarrollado por JetBrains. Es interoperable con Java y se ha convertido en el lenguaje oficial para Android. Sus principales características incluyen la **inmutabilidad por defecto**, la **gestión segura de nulos**, el soporte para **programación funcional** (lambdas, funciones de orden superior) y **extensiones de clases**, lo que permite escribir código más limpio y eficiente.

3. Optional Chaining

El **Optional Chaining** en Kotlin permite acceder de manera segura a propiedades o métodos de un objeto que puede ser nulo. Se utiliza el operador `?.` para **evitar errores por referencia nula**.

Ejemplo:

```
val valor: String? = null
```

```
println(valor?.length) // Retorna null en lugar de lanzar una excepción
```

Este mecanismo evita fallos en tiempo de ejecución y mejora la seguridad del código frente a nulos.

4. Nullish Coalescing Operator (Elvis)

El **operador Elvis (?:)** permite asignar un valor predeterminado cuando una expresión puede ser nula. Se usa para simplificar estructuras condicionales y mantener la consistencia del flujo de datos.

```
var valor2: Int? = null
```

```
println(valor2 ?: 34) // Imprime 34 porque valor2 es null
```

El operador mejora la legibilidad y evita comprobaciones explícitas de null.

5. Listas mutables e inmutables

En Kotlin, las colecciones pueden ser:

- **Inmutables (listOf)**: no se pueden modificar después de su creación.
- **Mutables (mutableListOf)**: permiten agregar, eliminar o cambiar elementos.

Ejemplo:

```
val listaInmutable = listOf("Lunes", "Martes")
val listaMutable = mutableListOf("Lunes", "Martes")
listaMutable.add("Miércoles")
```

El uso de listas inmutables promueve seguridad y evita efectos colaterales no deseados.

6. Operaciones sobre listas

Kotlin proporciona funciones para trabajar con listas de forma funcional y concisa:

- **Eliminación**: remove, removeAt.
- **Mapeo**: map transforma los elementos aplicando una función.
- **Combinación**: zip une dos listas en pares.
- **Aplanamiento**: flatten convierte listas de listas en una sola lista.
- **Filtrado**: filter selecciona elementos que cumplen una condición.
- **Ordenamiento**: sorted, sortedBy ordena elementos según un criterio.

Estas operaciones permiten manipular datos de manera eficiente y expresiva.

7. Sobrecarga de funciones

La **sobrecarga de funciones** consiste en definir varias funciones con el mismo nombre pero con diferente número o tipo de parámetros. Permite **reutilizar nombres de funciones** para operaciones relacionadas y mejorar la legibilidad.

```
fun sumar(a: Int, b: Int) = a + b
fun sumar(a: Int, b: Int, c: Int) = a + b + c
```

8. Lambda

Las **lambdas** son funciones anónimas que se pueden asignar a variables o pasar como parámetros. Permiten escribir código **conciso y funcional**, ideal para operaciones sobre colecciones o callbacks.

```
val restar: (Int) -> Int = { x -> x - 2 }
println(restar(5)) // Imprime 3
```

9. forEach

forEach es una función de extensión que recorre cada elemento de una colección y ejecuta una lambda sobre él. Es una alternativa más legible al bucle for tradicional.

```
val lista = listOf("Juan", "Pedro")
lista.forEach { println(it) }
```

10. vararg

vararg permite que una función reciba un **número variable de argumentos**. Es útil cuando no se conoce de antemano la cantidad de parámetros a recibir.

```
fun comida(vararg platos: String) {  
    platos.forEach { println(it) }  
}
```

11. Excepciones

Kotlin maneja errores mediante **excepciones** (try-catch-finally). Permite **capturar errores en tiempo de ejecución** y mantener la ejecución del programa.

```
try {  
    val nombre: String? = null  
    println(nombre!!.length)  
} catch (e: Exception) {  
    println("Ocurrió un error: $e")  
} finally {  
    println("Fin del programa")  
}
```

- try → bloque donde puede ocurrir un error.
- catch → bloque que maneja la excepción.
- finally → bloque que siempre se ejecuta, útil para liberar recursos.

Desarrollo

PRÁCTICA 1

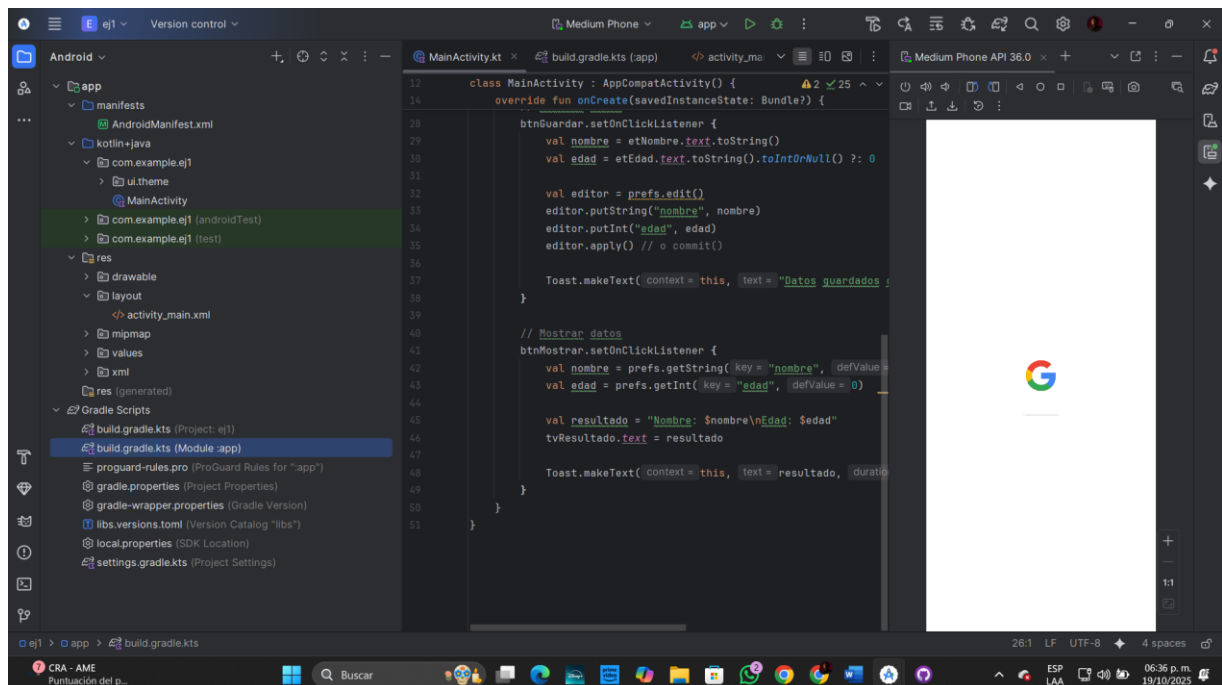
Para esta practica tenemos que igualmente abrir un nuevo proyecto el cual daremos paso a integrar paso a paso lo que es lo siguiente:

- Dos EditText: uno para nombre y otro para edad.
- Un botón para leer
- Otro botón para insertar

Implementar el código Kotlin:

- Al presionar “Guardar”, usar `getSharedPreferences("datos", MODE_PRIVATE)` y guardar nombre y edad.
- Al presionar “Mostrar”, recuperar los datos guardados y mostrarlos en un Toast o TextView.

Para este igual aremos lo que es nombrar el nombre del proyecto en mi caso le puse ej1 de aquí lo primero que aremos es en la parte de la carpeta “res” la abriremos y nos aparecerán otras carpetas de ahí nosotros crearemos una que le pondremos layout de ahí crearemos el `activity.main.xml` el cual le dará una forma de lo que estamos buscando como se muestra a continuación la ubicación del layout:



De aquí observaremos como se muestra 2 recuadros completamente vacíos esto es porque nos falta generar todo lo que va a llevar lo que yo are será ponerlo directamente con código para esto utilice esto:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/etNombre"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nombre"
        android:inputType="textPersonName" />

    <EditText
        android:id="@+id/etEdad"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Edad"
        android:inputType="number" />

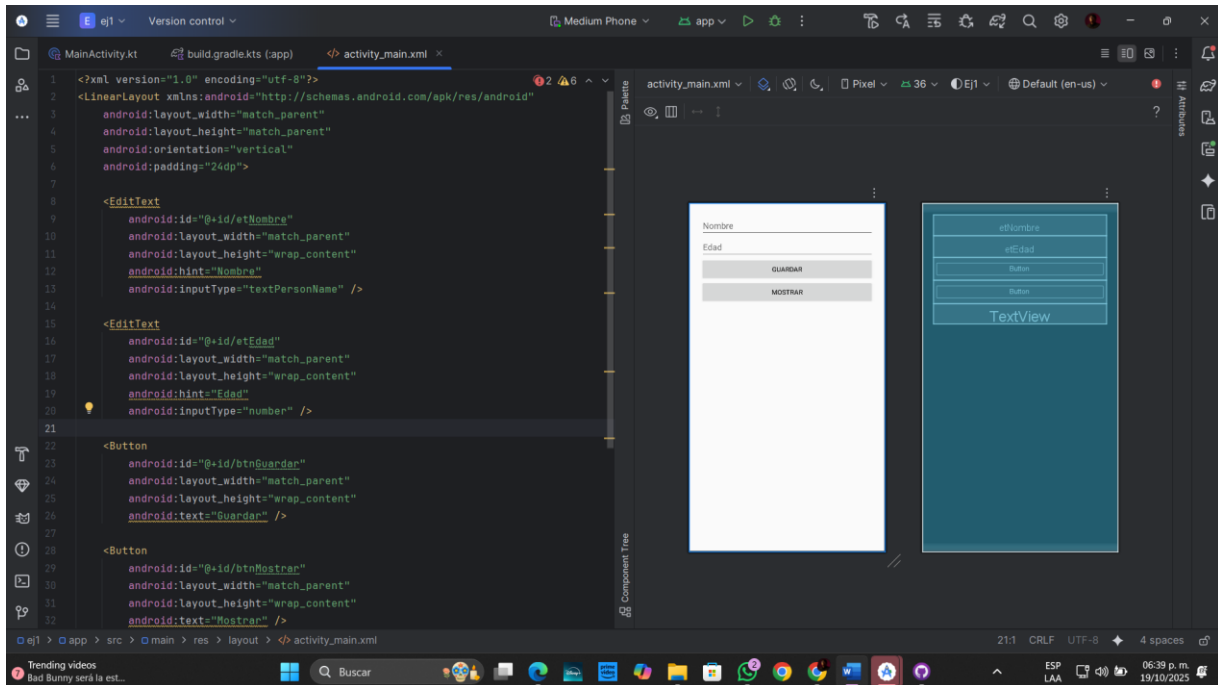
    <Button
        android:id="@+id/btnGuardar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Guardar" />

    <Button
        android:id="@+id/btnMostrar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Mostrar" />

    <TextView
        android:id="@+id/tvResultado"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text=""
        android:paddingTop="20dp"
        android:textSize="18sp" />

</LinearLayout>
```

Este nos dará un resultado así dentro de los recuadros



De aquí nos iremos a nuestro main que el cual vienen paqueterías por defecto en mi caso yosolo utilice estos en especial:

```
import android.content.Context
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
```

Esto es lo que hace cada importación:

- Context → para acceder a SharedPreferences.
- Bundle → para manejar el ciclo de vida de la actividad.
- Button, EditText, TextView, Toast → elementos de la interfaz.
- AppCompatActivity → clase base para actividades en Android.
- enableEdgeToEdge() → hace que la interfaz se muestre debajo de la barra de estado (estilo moderno de Android 14/SDK 35).

De aquí pues aremos que declararemos la clase principal con el “class MainActivity : AppCompatActivity() { “

Después de aran tanto el onCreate como el setContentView que nos ara lo siguiente onCreate() se ejecuta cuando la app **inicia la pantalla**.

setContentView(R.layout.activity_main) carga el diseño de la interfaz (activity_main.xml).

Aquí conectas los **elementos visuales** del XML con variables Kotlin. Así puedes manipularlos (leer, escribir, etc.) desde el código.

```
val etNombre = findViewById<EditText>(R.id.etNombre)
val etEdad = findViewById<EditText>(R.id.etEdad)
val btnGuardar = findViewById<Button>(R.id.btnGuardar)
val btnMostrar = findViewById<Button>(R.id.btnMostrar)
val tvResultado = findViewById<TextView>(R.id.tvResultado)
```

```
val prefs = getSharedPreferences("datos", Context.MODE_PRIVATE)
```

Crea un archivo llamado “datos” (por defecto se guarda en el almacenamiento interno de la app).

MODE_PRIVATE significa que solo esta aplicación puede acceder a ese archivo.

Lo siguiente nos ara guardar datos

```
btnGuardar.setOnClickListener {
    val nombre = etNombre.text.toString()
    val edad = etEdad.text.toString().toIntOrNull() ?: 0
```

Obtiene los valores escritos en los EditText.

toIntOrNull() convierte el texto a número y, si no es válido, usa 0.

Luego se crea un **editor** para guardar los datos:

```
val editor = prefs.edit()
editor.putString("nombre", nombre)
editor.putInt("edad", edad)
editor.apply()
```

- prefs.edit() → abre el editor del archivo.
- putString() y putInt() → guardan los valores.
- apply() → guarda los cambios de forma asíncrona (sin bloquear la app).

Ya por ultimo mostrara los resultados guardados:

```
btnMostrar.setOnClickListener {
    val nombre = prefs.getString("nombre", "No disponible")
    val edad = prefs.getInt("edad", 0)
```

- Recupera los valores guardados.
- Si no existen, muestra los valores por defecto ("No disponible" y 0).

```
val resultado = "Nombre: $nombre\nEdad: $edad"
tvResultado.text = resultado
```

```
Toast.makeText(this, resultado, Toast.LENGTH_LONG).show()
```

```
}
```

- Escribe los datos en el TextView.
- También los muestra con un Toast (ventana emergente temporal).

```

1 package com.example.ej1
2
3 import android.content.Context
4 import android.os.Bundle
5 import android.widget.Button
6 import android.widget.EditText
7 import android.widget.TextView
8 import android.widget.Toast
9 import androidx.activity.enableEdgeToEdge
10 import androidx.appcompat.app.AppCompatActivity
11
12 // Usage
13 class MainActivity : AppCompatActivity() {
14
15     override fun onCreate(savedInstanceState: Bundle?) {
16         super.onCreate(savedInstanceState)
17         enableEdgeToEdge()
18         setContentView(R.layout.activity_main)
19
20         val etNombre = findViewById<EditText>(R.id.etNombre)
21         val etEdad = findViewById<EditText>(R.id.etEdad)
22         val btnGuardar = findViewById<Button>(R.id.btnGuardar)
23         val btnMostrar = findViewById<Button>(R.id.btnMostrar)
24         val tvResultado = findViewById<TextView>(R.id.tvResultado)
25
26         val prefs = getSharedPreferences("datos", Context.MODE_PRIVATE)
27
28         // Guardar datos
29         btnGuardar.setOnClickListener {
30             val nombre = etNombre.text.toString()
31             val edad = etEdad.text.toString().toIntOrNull() ?: 0
32
33             val editor = prefs.edit()
34             editor.putString("nombre", nombre)
35             editor.putInt("edad", edad)
36             editor.apply() // o commit()
37
38             Toast.makeText(context = this, text = "Datos guardados correctamente", duration = Toast.LENGTH_SHORT).show()
39         }
40
41         // Mostrar datos
42         btnMostrar.setOnClickListener {
43             val nombre = prefs.getString(key = "nombre", defValue = "No disponible")
44             val edad = prefs.getInt(key = "edad", defValue = 0)
45
46             val resultado = "Nombre: $nombre\nEdad: $edad"
47             tvResultado.text = resultado
48
49             Toast.makeText(context = this, text = resultado, duration = Toast.LENGTH_LONG).show()
50         }
51     }
52 }

```

Igualmente, esta como comentario que es lo que hace cada cosa

PRÁCTICA 2:

Para el siguiente punto en mi caso hice un proyecto a parte por lo mismo para que no llegue a causar alguna contradicción, pero igual los mismos pasos, pero con los siguientes puntos a tratar.

- Un EditText multiline para capturar texto.
- Un botón (escribe texto en archivo).
- Un botón (lee contenido del archivo).
- Mostrar el contenido leído en un TextView.

Importaciones:

```
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import java.io.FileInputStream
import java.io.FileOutputStream
import java.io.IOException
```

al igual que el ejercicio 1 es casi parte de lo mismo pero lo que lo diferencia del primero y segundo son las variables que ponemos como diré a continuación:

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
```

en este veremos tanto para el texto, guardar, leer y el contenido dando el siguiente punto del código

```
        val etTexto = findViewById<EditText>(R.id.etTexto)
        val btnGuardar = findViewById<Button>(R.id.btnGuardar)
        val btnLeer = findViewById<Button>(R.id.btnLeer)
        val tvContenido = findViewById<TextView>(R.id.tvContenido)

        val nombreArchivo = "nota.txt"
```

Antes de seguir tenemos que decir sobre los métodos de FileOutputStream y SharedPreferences mas que nada para ver que es lo que nos conviene dentro del programa

El método `openFileOutput("nota.txt", MODE_PRIVATE)` crea o abre el archivo **nota.txt** dentro del almacenamiento interno de la aplicación, y el modo `MODE_PRIVATE` indica que solo esa app puede acceder a dicho archivo. Luego, `fileOutput.write(texto.toByteArray())` convierte el texto ingresado a bytes y los escribe dentro del archivo, permitiendo guardar la información de forma persistente. Por su parte, `fileInput.bufferedReader().use { it.readText() }` abre el archivo para lectura y obtiene todo su contenido como texto, mostrando los datos previamente guardados. Finalmente, los datos permanecen almacenados en el directorio interno de la aplicación, lo que garantiza su persistencia incluso al cerrar la app.

```
// Leer texto del archivo
btnLeer.setOnClickListener {
    try {
        val fileInput: FileInputStream = openFileInput(nombreArchivo)
        val contenido = fileInput.bufferedReader().use { it.readText() }
        fileInput.close()
        tvContenido.text = contenido
        Toast.makeText(this, "Texto leído correctamente",
Toast.LENGTH_SHORT).show()
    } catch (e: IOException) {
        e.printStackTrace()
        Toast.makeText(this, "Error al leer el archivo", Toast.LENGTH_SHORT).show()
    }
}
}
```

La diferencia entre:

SharedPreferences

- Está pensado para guardar datos pequeños y simples, como configuraciones o preferencias del usuario (por ejemplo, nombre, edad, modo oscuro, sesión activa, etc.).
- Guarda la información en pares clave–valor (como un diccionario o mapa).
- Los datos se almacenan automáticamente en un archivo XML interno que maneja Android.
- Ideal cuando necesitas leer o escribir pocos valores rápidamente.

FileStream (FileOutputStream / FileInputStream)

- Se utiliza para guardar texto o información más extensa, como notas, reportes, listas o cualquier bloque de texto.
- Tú decides cómo se escribe y se lee el contenido del archivo.
- Trabaja directamente con archivos de texto (.txt) en el almacenamiento interno.
- Ideal cuando necesitas guardar grandes cantidades de texto o datos estructurados manualmente.

The image displays two sequential screenshots of the Android Studio IDE, showing the development of a Kotlin-based Android application. The top screenshot shows the initial setup of the MainActivity class, including imports for Android OS, Widgets, and File I/O. The bottom screenshot shows the implementation of the onCreate method, which includes saving text to a file and reading it back.

```

package com.example.ej2

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import java.io.FileInputStream
import java.io.FileOutputStream
import java.io.IOException

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        val etTexto = findViewById<EditText>(R.id.etTexto)
        val btnGuardar = findViewById<Button>(R.id.btnGuardar)
        val btnLeer = findViewById<Button>(R.id.btnLeer)
        val tvContenido = findViewById<TextView>(R.id.tvContenido)

        val nombreArchivo = "nota.txt"

        // Guardar texto en archivo
        btnGuardar.setOnClickListener {
            val texto = etTexto.text.toString()

            try {
                val fileOutput: FileOutputStream = openFileOutput(nombreArchivo, MODE_PRIVATE)
                fileOutput.write(texto.toByteArray())
                fileOutput.close()
                Toast.makeText(this, "Texto guardado correctamente", duration = Toast.LENGTH_SHORT).show()
                etTexto.text.clear()
            } catch (e: IOException) {
                e.printStackTrace()
                Toast.makeText(this, "Error al guardar", duration = Toast.LENGTH_SHORT).show()
            }
        }

        // Leer texto del archivo
        btnLeer.setOnClickListener {
            try {
                val fileInput: FileInputStream = openFileInput(nombreArchivo)
                val contenido = fileInput.bufferedReader().use { it.readText() }
                fileInput.close()
                tvContenido.text = contenido
                Toast.makeText(this, "Texto leído correctamente", duration = Toast.LENGTH_SHORT).show()
            } catch (e: IOException) {
                e.printStackTrace()
                Toast.makeText(this, "Error al leer el archivo", duration = Toast.LENGTH_SHORT).show()
            }
        }
    }
}

```

Para la siguiente parte hablaremos del activity_main.xml:

Creamos un diseño con un EditText grande (multilínea), dos botones y un TextView para mostrar el contenido leído.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/etTexto"
        android:layout_width="match_parent"
        android:layout_height="200dp"
        android:gravity="top"
        android:hint="Escribe algo aquí..."
        android:inputType="textMultiLine"
        android:scrollbars="vertical" />

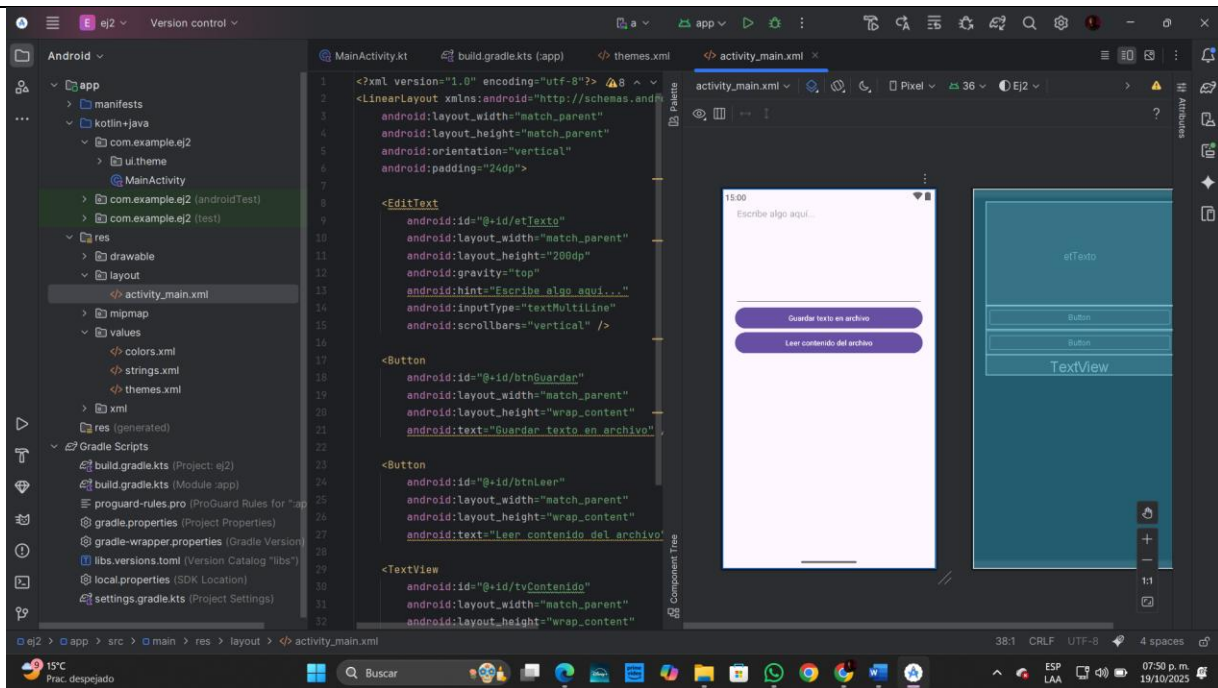
    <Button
        android:id="@+id/btnGuardar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Guardar texto en archivo" />

    <Button
        android:id="@+id/btnLeer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Leer contenido del archivo" />

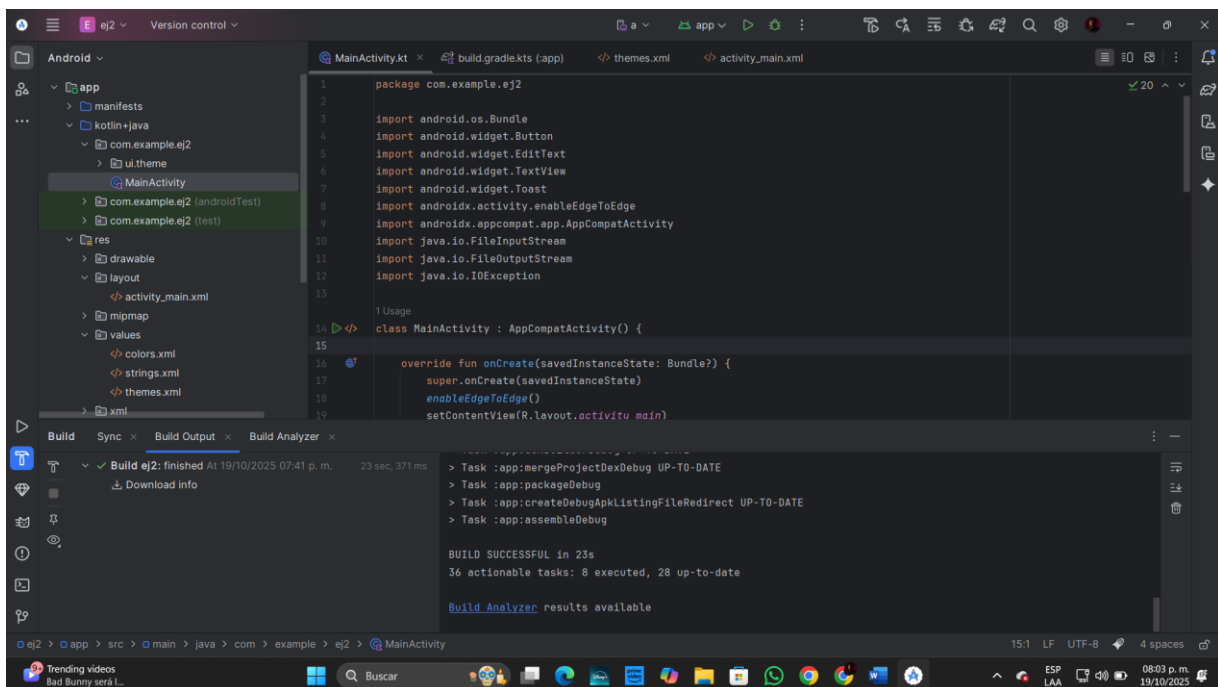
    <TextView
        android:id="@+id/tvContenido"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:paddingTop="16dp"
        android:textSize="16sp"
        android:textColor="@android:color/black" />

</LinearLayout>
```

De aquí ya sería cuestión de ver que tanto se ve dentro del .xml



Y al correrlo no nos dará ningún tipo de error:



PRÁCTICA 3:

En este nos piden lo siguiente:

Crear una base de datos SQLite en Android usando un modelo DAO, implementando operaciones CRUD: crear, leer, actualizar y eliminar registros.

MATERIAL Y SOFTWARE

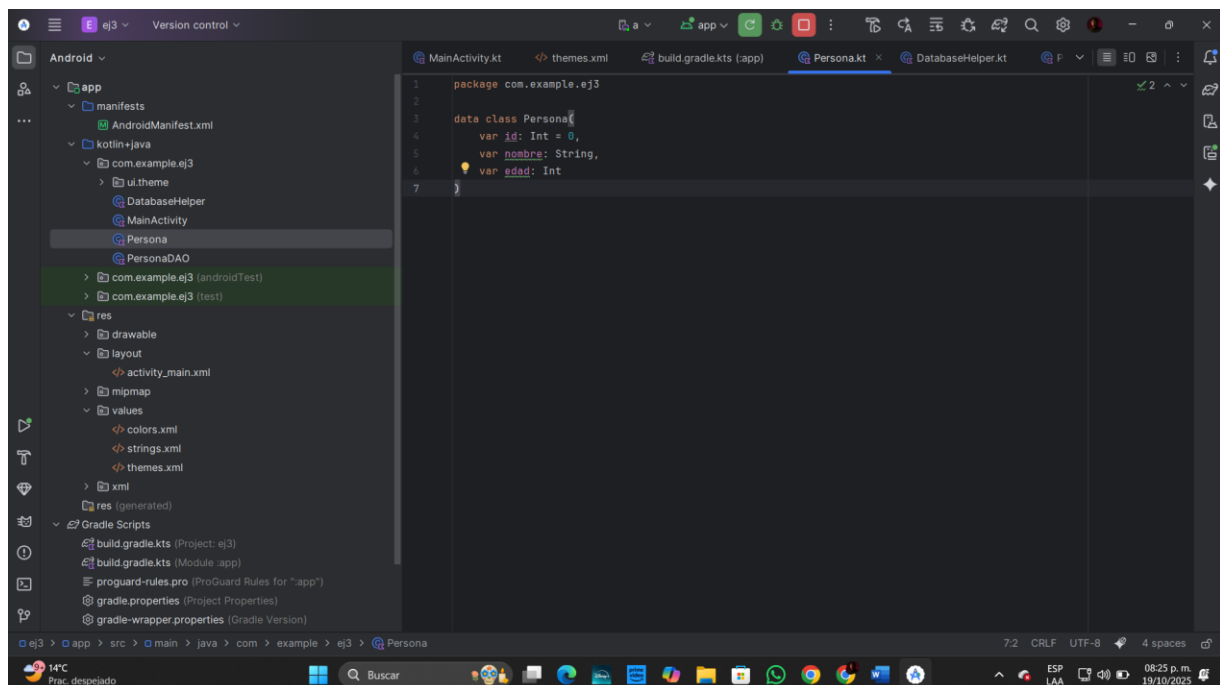
Crear un proyecto y diseñar la interfaz con campos para realizar DAO, además de los botones. Implementar cada operación usando los métodos del DAO.

Okay para este mismo igual abriré otro proyecto como ej3 para que se vea diferente cada uno igual se crearán los mismos puntos del layout y la ejecución pero en este tenemos que checar los métodos que usaremos y las clases ahora como mostraremos primero tenemos que implementar el modelo de persona, database, DAO (es el acceso a los objetos)

Todo esto estará dentro de donde se encuentra el main activity

Primero el que le pondré persona:

Representa una persona con un identificador, un nombre y edad

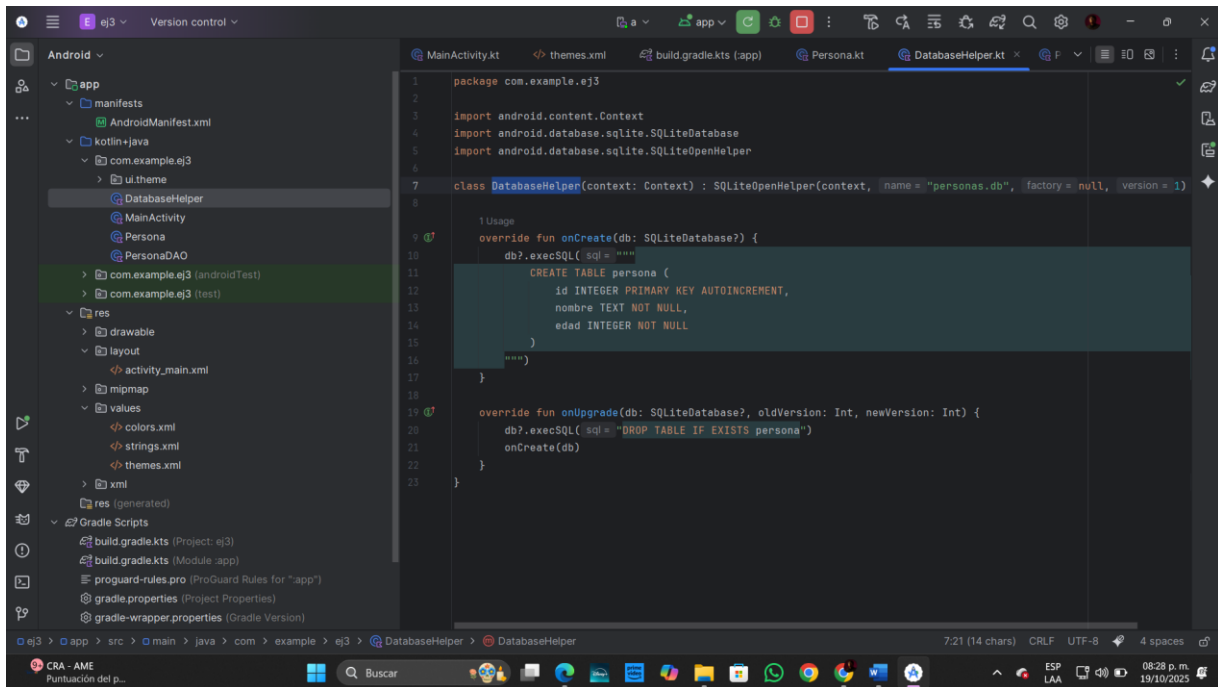


package com.example.ej3

```
data class Persona(
    var id: Int = 0,
    var nombre: String,
    var edad: Int
)
```

El siguiente es DatabaseHelper

Esta clase crea y administra la base de datos.



package com.example.ej3

import android.content.Context

import android.database.sqlite.SQLiteDatabase

import android.database.sqlite.SQLiteOpenHelper

class DatabaseHelper(context: Context) : SQLiteOpenHelper(context, "personas.db", null, 1)
{

```

    override fun onCreate(db: SQLiteDatabase?) {
        db?.execSQL("""
            CREATE TABLE persona (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                nombre TEXT NOT NULL,
                edad INTEGER NOT NULL
            )
        """)
    }
}
    
```

```

    override fun onUpgrade(db: SQLiteDatabase?, oldVersion: Int, newVersion: Int) {
        db?.execSQL("DROP TABLE IF EXISTS persona")
        onCreate(db)
    }
}
    
```

El siguiente es PersonaDAO.kt
Define los métodos del CRUD.

```

1 package com.example.ej3
2
3 import android.content.ContentValues
4 import android.content.Context
5
6 class PersonaDAO(context: Context) {
7     4 Usages
8     private val dbHelper = DatabaseHelper(context)
9
10    fun insertar(persona: Persona): Long {
11        val db = dbHelper.writableDatabase
12        val values = ContentValues().apply {
13            put("nombre", persona.nombre)
14            put("edad", persona.edad)
15        }
16        val id = db.insert(table = "persona", nullColumnHack = null, values)
17        db.close()
18        return id
19    }
20
21    fun obtenerTodos(): List<Persona> {
22        val lista = mutableListOf<Persona>()
23        val db = dbHelper.readableDatabase
24        val cursor = db.rawQuery("SELECT * FROM persona", selectionArgs = null)
25        if (cursor.moveToFirst()) {
26            do {
27                val persona = Persona(
28                    id = cursor.getInt(columnIndex = 0),
29                    nombre = cursor.getString(columnIndex = 1),
30                    edad = cursor.getInt(columnIndex = 2)
31                )
32                lista.add(persona)
33            } while (cursor.moveToNext())
34        }
35        cursor.close()
36    }
37 }
  
```

package com.example.ej3

import android.content.ContentValues

import android.content.Context

class PersonaDAO(context: Context) {
 private val dbHelper = DatabaseHelper(context)

```

fun insertar(persona: Persona): Long {
    val db = dbHelper.writableDatabase
    val values = ContentValues().apply {
        put("nombre", persona.nombre)
        put("edad", persona.edad)
    }
    val id = db.insert("persona", null, values)
    db.close()
    return id
}
  
```

```

fun obtenerTodos(): List<Persona> {
    val lista = mutableListOf<Persona>()
    val db = dbHelper.readableDatabase
  
```

```

val cursor = db.rawQuery("SELECT * FROM persona", null)
if (cursor.moveToFirst()) {
    do {
        val persona = Persona(
            id = cursor.getInt(0),
            nombre = cursor.getString(1),
            edad = cursor.getInt(2)
        )
        lista.add(persona)
    } while (cursor.moveToNext())
}
cursor.close()
db.close()
return lista
}

fun actualizar(persona: Persona): Int {
    val db = dbHelper.writableDatabase
    val values = ContentValues().apply {
        put("nombre", persona.nombre)
        put("edad", persona.edad)
    }
    val filas = db.update("persona", values, "id = ?", arrayOf(persona.id.toString()))
    db.close()
    return filas
}

fun eliminar(id: Int): Int {
    val db = dbHelper.writableDatabase
    val filas = db.delete("persona", "id = ?", arrayOf(id.toString()))
    db.close()
    return filas
}
}

```

ahora por ultimo el mainActivity

El MainActivity **usa** una clase llamada PersonaDAO, la cual se encarga de hablar con la base de datos SQLite (por eso se llama *DAO: Data Access Object*).

```
package com.example.ej3
```

```

import android.os.Bundle
import android.widget.*
import androidx.appcompat.app.AppCompatActivity

```

```
class MainActivity : AppCompatActivity() {
```

```
private lateinit var dao: PersonaDAO
private lateinit var etId: EditText
private lateinit var etNombre: EditText
private lateinit var etEdad: EditText
private lateinit var tvResultado: TextView

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    dao = PersonaDAO(this)

    etId = findViewById(R.id.etId)
    etNombre = findViewById(R.id.etNombre)
    etEdad = findViewById(R.id.etEdad)
    tvResultado = findViewById(R.id.tvResultado)

    findViewById<Button>(R.id.btnInsertar).setOnClickListener {
        val nombre = etNombre.text.toString()
        val edad = etEdad.text.toString().toIntOrNull() ?: 0
        dao.insertar(Persona(nombre = nombre, edad = edad))
        Toast.makeText(this, "Insertado correctamente", Toast.LENGTH_SHORT).show()
    }

    findViewById<Button>(R.id.btnMostrar).setOnClickListener {
        val personas = dao.obtenerTodos()
        tvResultado.text = personas.joinToString("\n") { "ID:${it.id} - ${it.nombre} (${it.edad})" }
    }

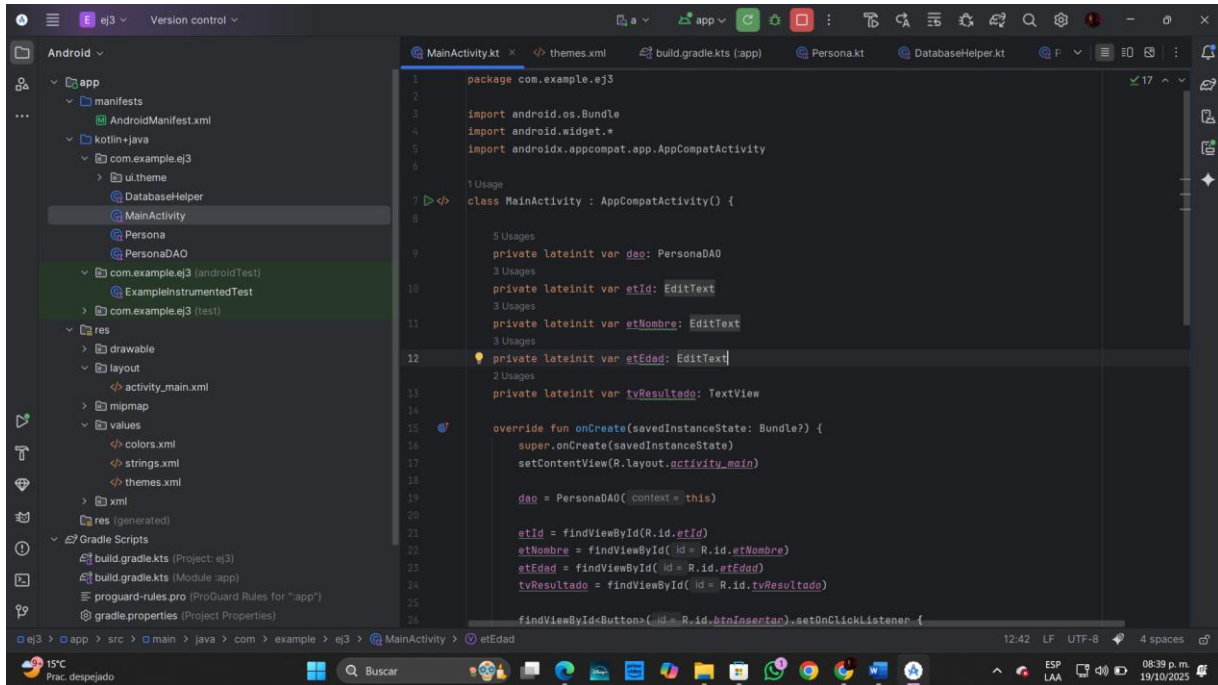
    findViewById<Button>(R.id.btnActualizar).setOnClickListener {
        val id = etId.text.toString().toIntOrNull()
        if (id != null) {
            val nombre = etNombre.text.toString()
            val edad = etEdad.text.toString().toIntOrNull() ?: 0
            dao.actualizar(Persona(id, nombre, edad))
            Toast.makeText(this, "Actualizado correctamente",
Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this, "Ingresa un ID válido", Toast.LENGTH_SHORT).show()
        }
    }

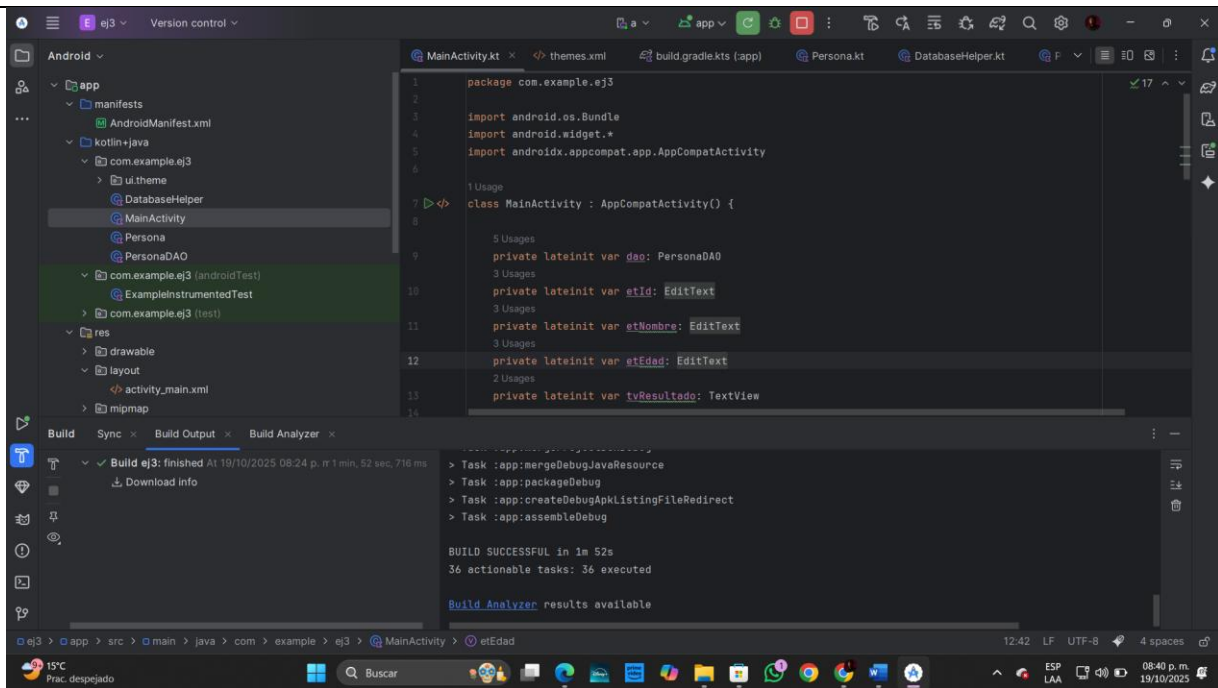
    findViewById<Button>(R.id.btnEliminar).setOnClickListener {
        val id = etId.text.toString().toIntOrNull()
        if (id != null) {
```

```

        dao.eliminar(id)
        Toast.makeText(this, "Eliminado correctamente", Toast.LENGTH_SHORT).show()
    } else {
        Toast.makeText(this, "Ingresa un ID válido", Toast.LENGTH_SHORT).show()
    }
}
}
}
}

```





En este caso solo es para verificar en cuando funciona correctamente sin ningún error
Por último es el activity_main.xml:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:gravity="center_horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<EditText
    android:id="@+id/etId"
    android:hint="ID (solo para editar/eliminar)"
    android:inputType="number"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
```

```
<EditText
    android:id="@+id/etNombre"
    android:hint="Nombre"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
```

```
<EditText
    android:id="@+id/etEdad"
    android:hint="Edad"
```



```
android:inputType="number"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>
```

```
<Button android:id="@+id/btnInsertar" android:text="Insertar"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
```

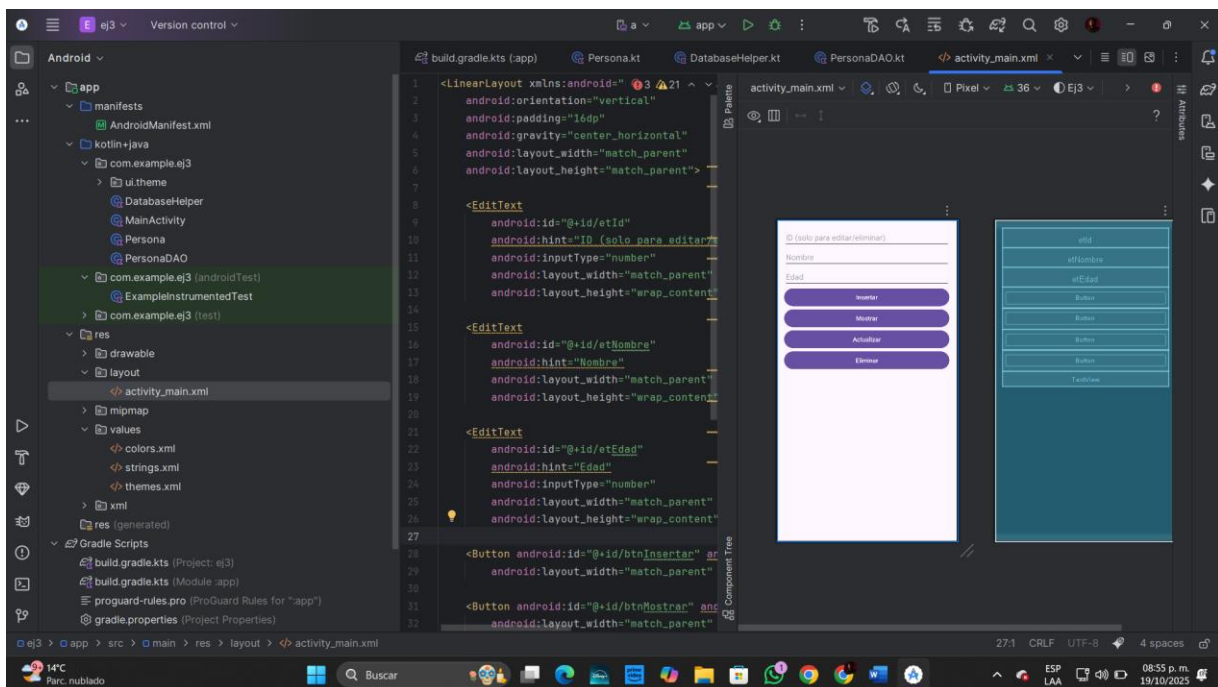
```
<Button android:id="@+id/btnMostrar" android:text="Mostrar"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
```

```
<Button android:id="@+id/btnActualizar" android:text="Actualizar"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
```

```
<Button android:id="@+id/btnEliminar" android:text="Eliminar"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
```

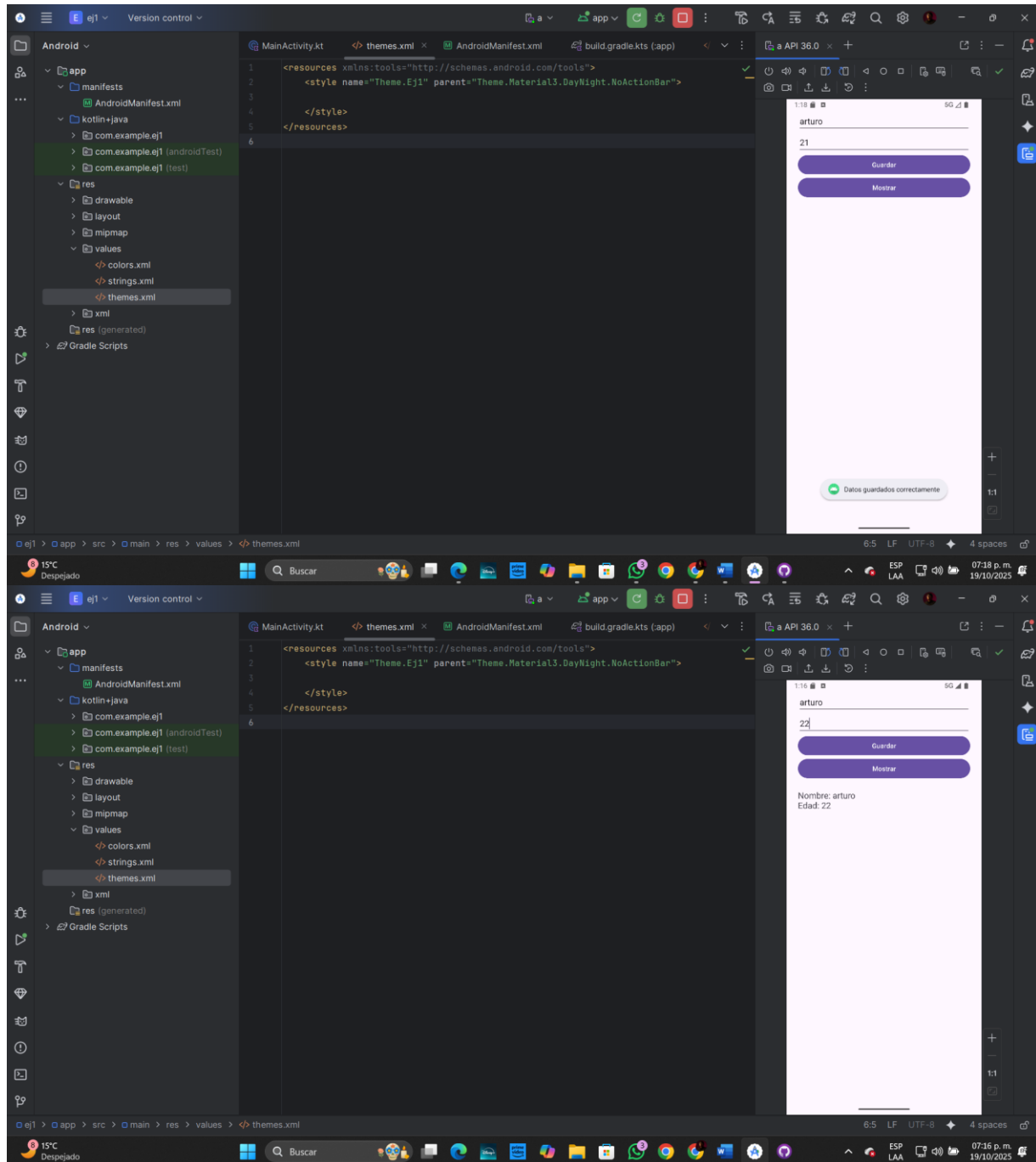
```
<TextView android:id="@+id/tvResultado"
    android:text=""
    android:padding="8dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
```

```
</LinearLayout>
```

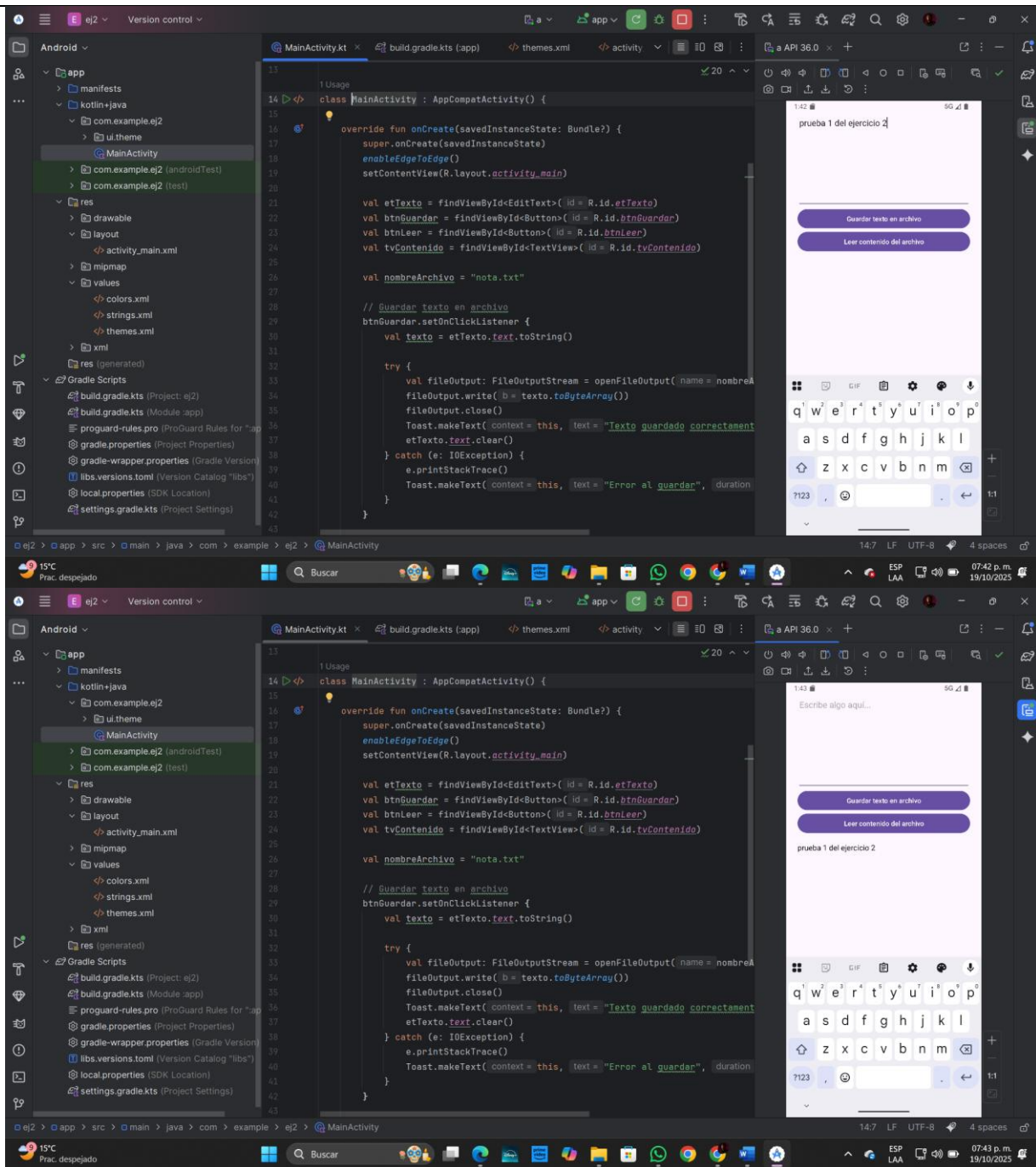


Resultados

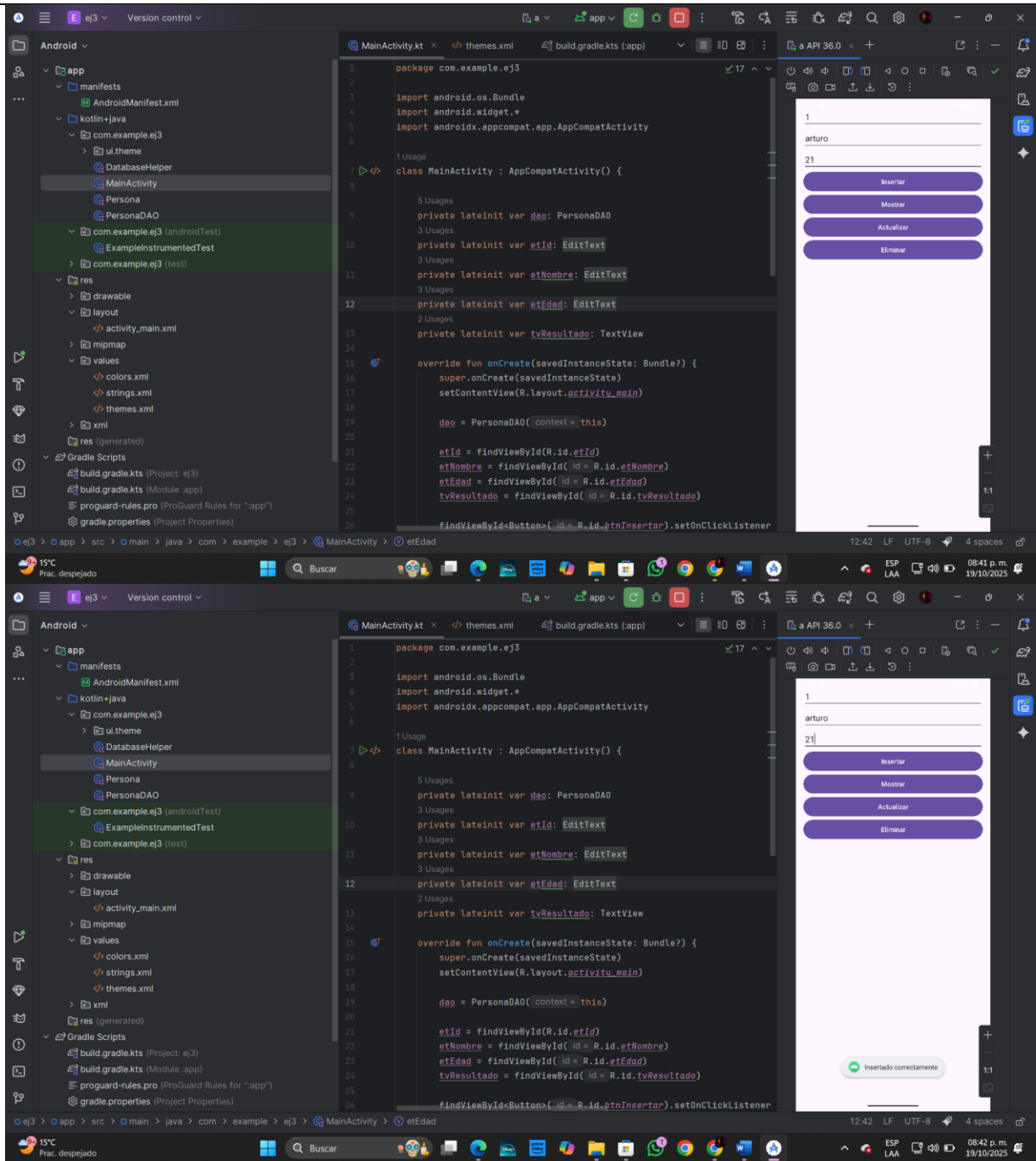
Practica1:



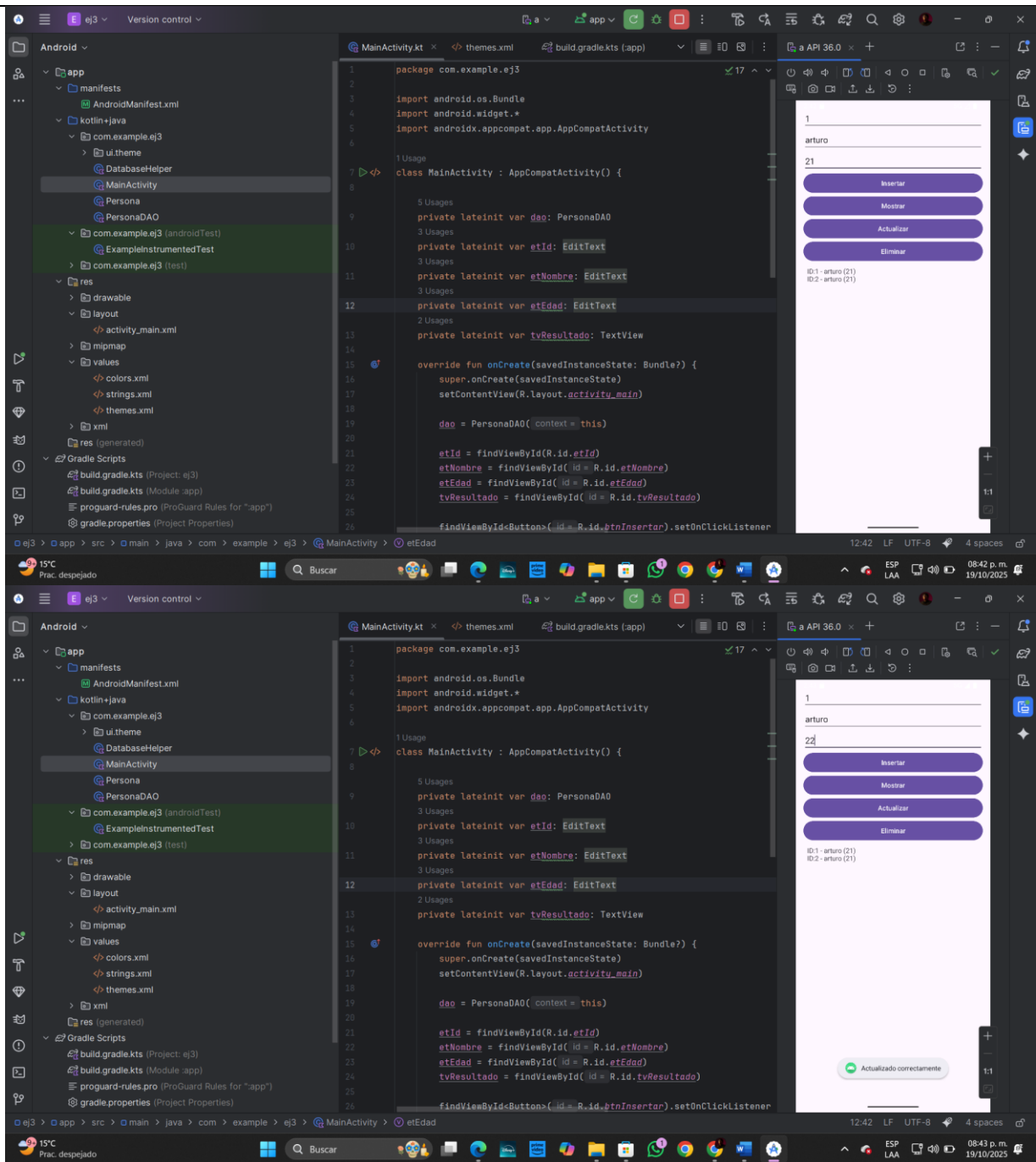
Practica2:



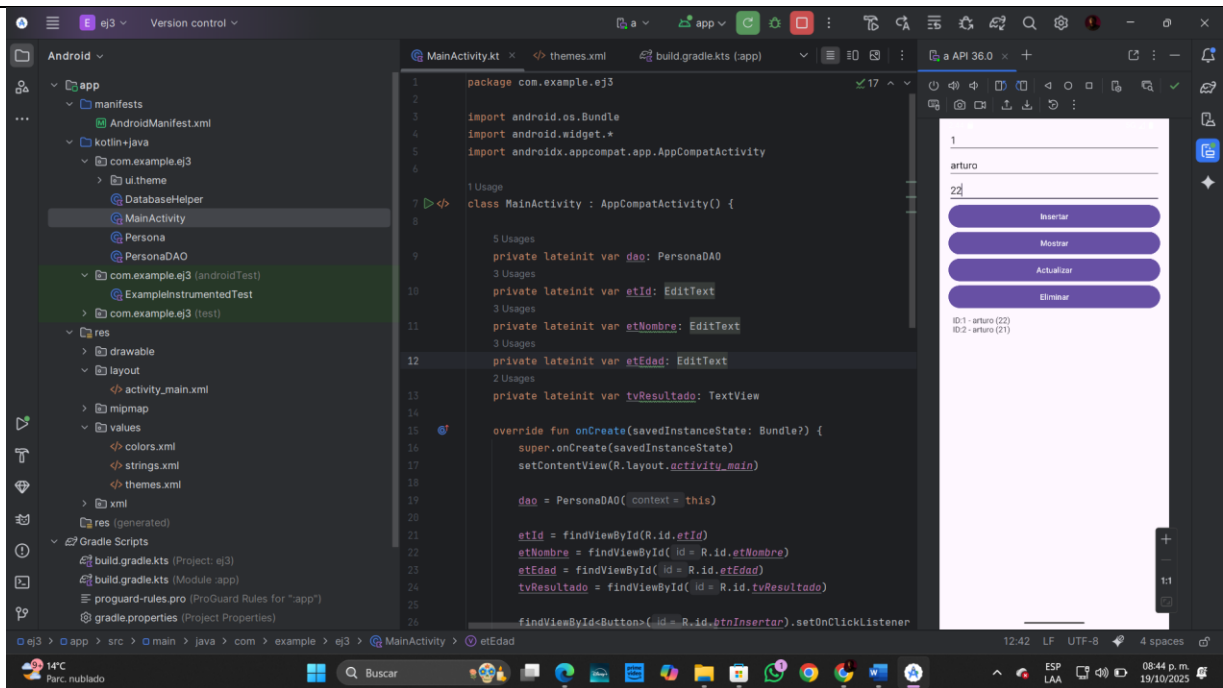
Practica3:
insertar



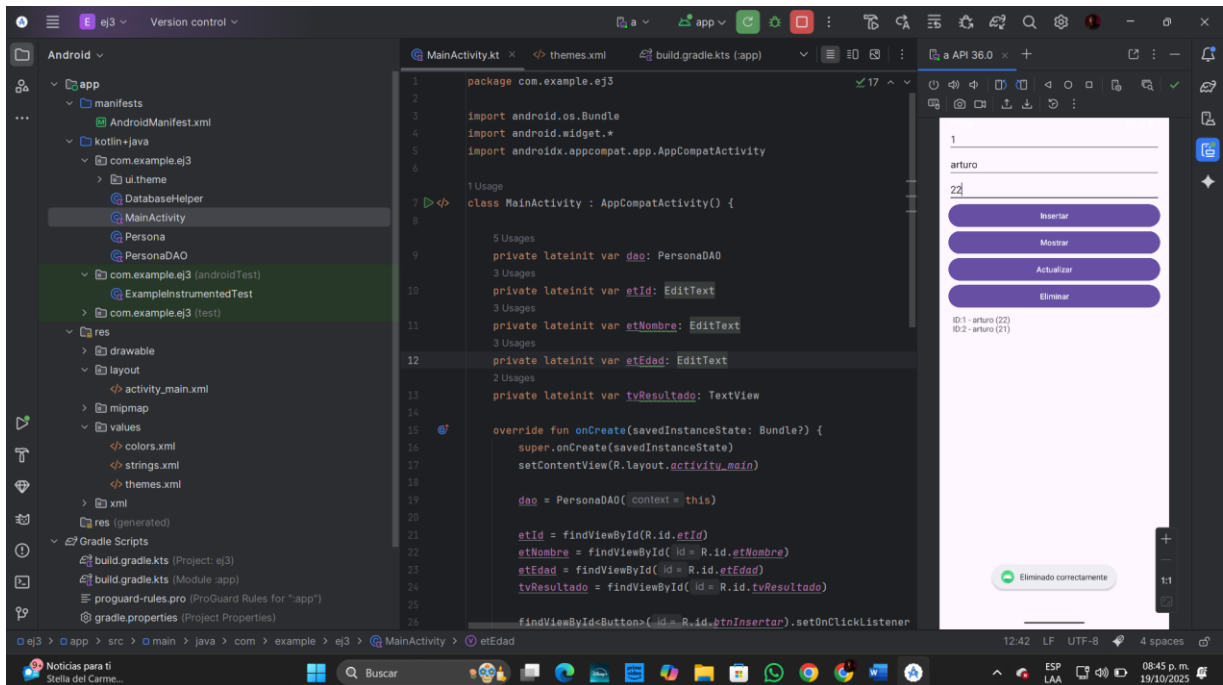
mostrar

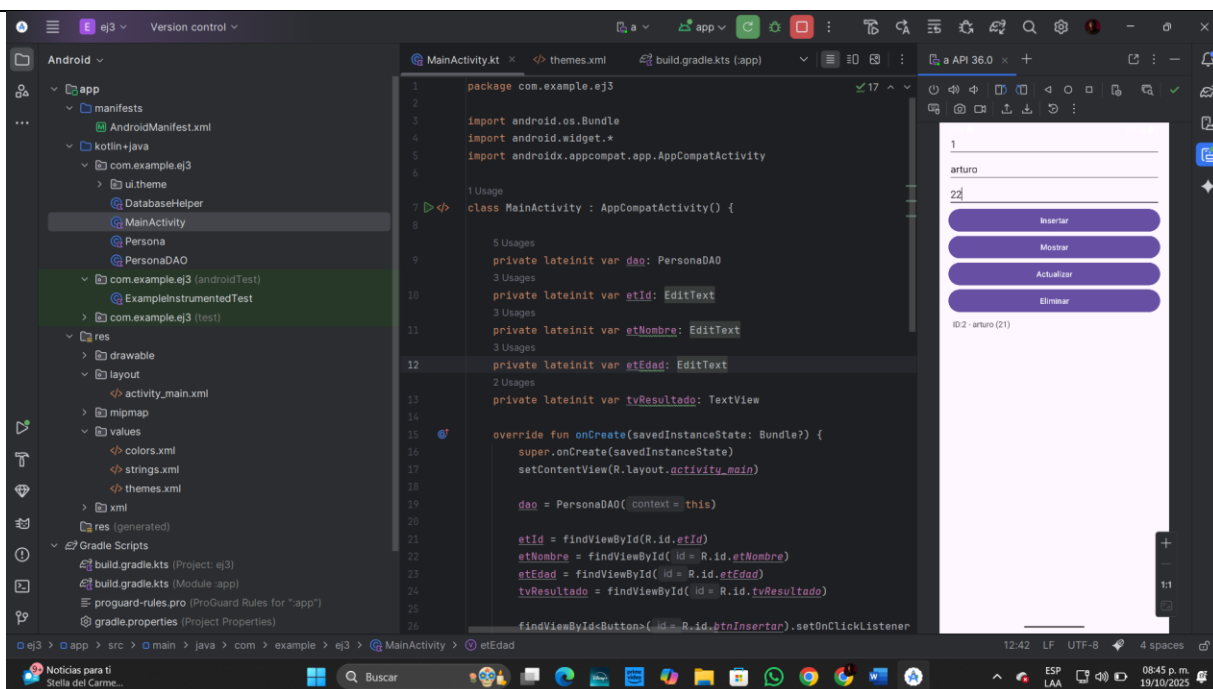


Actualizar:



Eliminar:





Conclusión general:

En estas tres prácticas se abordó la **persistencia de datos en Android usando Kotlin**, explorando diferentes métodos según la complejidad y el tipo de información a almacenar.

La **Práctica 1** con **SharedPreferences** permitió guardar y recuperar datos primitivos como cadenas, números y valores booleanos de manera rápida y sencilla. Esta técnica es útil para información ligera, como configuraciones de usuario o preferencias, garantizando que los datos persistan al cerrar la aplicación.

La **Práctica 2** implementó la **lectura y escritura en archivos locales** mediante `FileOutputStream` y `FileInputStream`. Esto permitió manejar texto de forma más libre y dinámica, almacenando y recuperando contenidos completos, como notas o registros que no se limitan a un tipo de dato específico. La información también se mantiene aunque la app se cierre, aprovechando el almacenamiento interno seguro de Android.

Finalmente, la **Práctica 3** introdujo el **uso de SQLite con el patrón DAO**, implementando un sistema CRUD completo (crear, leer, actualizar y eliminar). Esta práctica mostró cómo gestionar datos estructurados de manera organizada, usando objetos (`Persona`) y una capa de acceso a datos (`PersonaDAO`) que separa la lógica de la base de datos de la interfaz de usuario. Es ideal para aplicaciones más complejas donde se necesitan almacenar múltiples registros con relaciones o realizar consultas avanzadas.

En conjunto, estas prácticas proporcionaron un entendimiento progresivo de las distintas formas de **persistir información en Android**, desde opciones simples y rápidas hasta soluciones más robustas y estructuradas, reforzando conceptos de **gestión de datos**, **manipulación de archivos** y **diseño limpio con patrones de programación**.

Referencias bibliográficas:

Android Developers. (s. f.). *Guía de Android Studio*. Recuperado de <https://developer.android.com/studio>

Android Developers. (s. f.). *Ciclo de vida de una actividad*. Recuperado de <https://developer.android.com/guide/components/activities/activity-lifecycle>

Android Developers. (s. f.). *RecyclerView Overview*. Recuperado de <https://developer.android.com/guide/topics/ui/layout/recyclerview>

Android Developers. (s. f.). *User Interface Layouts*. Recuperado de <https://developer.android.com/guide/topics/ui/declaring-layout>

JetBrains. (s. f.). *Kotlin Documentation*. Recuperado de <https://kotlinlang.org/docs/home.html>

W3C. (1998). *Extensible Markup Language (XML) 1.0*. World Wide Web Consortium. Recuperado de <https://www.w3.org/TR/xml/>

Google. (s. f.). *Android SDK*. Recuperado de <https://developer.android.com/studio/releases/sdk-tools>

Android Developers. (s. f.). *View Binding*. Recuperado de <https://developer.android.com/topic/libraries/view-binding>

Android Developers. (s. f.). *Data Binding Library*. Recuperado de <https://developer.android.com/topic/libraries/data-binding>

Google. (s. f.). *Best Practices for Android UI/UX Design*. Recuperado de <https://developer.android.com/design>

JetBrains. (s. f.). *Kotlin Standard Library Documentation*. Recuperado de <https://kotlinlang.org/api/latest/jvm/stdlib/>

JetBrains. (s. f.). *Kotlin Documentation*. Recuperado de <https://kotlinlang.org/docs/home.html>

Android Developers. (s. f.). *Kotlin Guides*. Recuperado de <https://developer.android.com/kotlin>

Android Developers. (s. f.). *SDK overview*. Recuperado de <https://developer.android.com/studio/releases/platforms>

Android Developers. (s. f.). *Build your first app*. Recuperado de <https://developer.android.com/codelabs/build-your-first-android-app> **JetBrains.** (s. f.). *Basic syntax*. Recuperado de <https://kotlinlang.org/docs/basicsyntax.html>